

# Efficient large scale machine learning training and inference on HPC systems

Full guide and checklist

# Version control

| Version | Date       | Changes                                                  | Author         |
|---------|------------|----------------------------------------------------------|----------------|
| 0.1     | 05-01-2026 | Initial document structure and outline of chapters       | Saamie Vincken |
| 0.2     | 07-01-2026 | Introduction and revisit document scope                  | Saamie Vincken |
| 0.3     | 09-01-2026 | GPU computation, CUDA, and mixed precision sections      | Saamie Vincken |
| 0.4     | 11-01-2026 | Data loading, CPU/GPU transfer, and performance sections | Saamie Vincken |
| 0.5     | 13-01-2026 | Memory usage, batch size, and inference sections         | Saamie Vincken |
| 0.6     | 15-01-2026 | Reproducibility, stability, and checkpointing            | Saamie Vincken |
| 0.7     | 16-01-2026 | HPC checklist and job configuration                      | Saamie Vincken |
| 1.0     | 17-01-2026 | Final review, cleanup, and document finalization         | Saamie Vincken |

## Distribution

| Version | Date       | Distributed to                                 | Status    |
|---------|------------|------------------------------------------------|-----------|
| 1.0     | 17-01-2026 | Canvas (Fontys University of Applied Sciences) | Submitted |
| 1.0     | 17-01-2026 | Slack (Institute for Data Science FHNW)        | Shared    |
| 1.0     | 17-01-2026 | Slack (Euclid Consortium)                      | Shared    |

# Table of Contents

|       |                                                                 |    |
|-------|-----------------------------------------------------------------|----|
| 1.    | <i>Introduction</i>                                             | 4  |
| 2.    | <i>Machine learning on HPC checklist</i>                        | 5  |
| 3.    | <i>Why HPC is needed for large scale ML work</i>                | 6  |
| 4.    | <i>GPU computation and why CUDA matters</i>                     | 6  |
| 5.    | <i>Mixed precision: doing less work without losing accuracy</i> | 7  |
| 6.    | <i>Parallel data loading</i>                                    | 8  |
| 7.    | <i>Keeping the CPU and GPU busy</i>                             | 9  |
| 8.    | <i>Reducing unnecessary work</i>                                | 10 |
| 8.1.  | Clearing gradients                                              | 10 |
| 8.2.  | Tracking and updating metrics                                   | 10 |
| 8.3.  | Logging and visualization                                       | 11 |
| 9.    | <i>Memory usage and batch size</i>                              | 12 |
| 10.   | <i>Job configuration</i>                                        | 13 |
| 10.1. | Scheduling                                                      | 13 |
| 11.   | <i>Inference at large scale</i>                                 | 14 |
| 12.   | <i>Reproducibility and stability</i>                            | 15 |
| 13.   | <i>The environment and conclusion</i>                           | 16 |

# 1. Introduction

High Performance Computing (HPC) systems are large systems that provide access to GPUs, large amounts of memory, fast storage, and multiple CPU cores. These systems are often used for machine learning because neural networks need to process massive datasets and perform a large volume of calculations that cannot be done efficiently on a single CPU or a normal computer.

Running machine learning code on an HPC system is different from running it on a local machine. Resources like GPUs, CPU cores and memory are factors that have a massive influence on how efficient a training script runs. Code that works locally can become slow or inefficient on an HPC system if it is not configured with the right setup.

This document is written for people who train or run machine learning models and need to execute them on GPU based HPC systems. It assumes basic knowledge of machine learning training and inference but does not assume experience with HPC systems.

The goal of this document is to explain how machine learning training and inference setups can be written in a way that works well on HPC systems. It focuses on training/inference code, data loading, memory usage, and job configurations.

This document can be used in two ways. The checklist in [Chapter 2](#) can be used to quickly check whether a training or inference setup is suitable for running on an HPC system. The chapters that follow explain each checklist item in detail, so that these choices can be understood and applied to different systems and workloads. Additionally, this document can be used for troubleshooting.

## 2. Machine learning on HPC checklist

The following checklist can be used by anyone needing to setup their training in a HPC environment. Complying with each of these checks means the training setup is optimized for a GPU based HPC environment in a reproducible, fast and memory efficient way. Each check point is explained in detail in the next chapters.

### **Main checks for GPU systems:**

- ☐ The model is moved to a CUDA device.
- ☐ All inputs are moved to the same device as the model.
- ☐ Batch size fits into available GPU memory.
- ☐ Gradients are cleared instead of set to zero.
- ☐ Training state and model is saved using checkpoints.
- ☐ Random seeds are set for reproducibility.
- ☐ Gradients are disabled during validation and inference.

### **Optional depending on resources:**

- ☐ Mixed precision is enabled.
- ☐ Data loading uses multiple workers.
- ☐ Number of workers matches available CPU cores.
- ☐ Pinned memory is enabled for data loading.
- ☐ GPU transfers are non-blocking.

### **Recommended for large scale runs:**

- ☐ Metrics are not computed every training step (only epoch wide).
- ☐ Logging and visualization are not done inside main training loop.

### 3. Why HPC is needed for large scale ML work

The latest machine learning models, such as (vision) transformers, convolutional neural networks and diffusion models, require processing millions of inputs and performing billions of floating point operations<sup>1</sup>. A single CPU cannot execute these operations in any reasonable time. High Performance Computing (HPC) systems solve this by providing access to GPU's, more memory, fast connections between resources, and shared storage.

However, simply running ML code on a HPC system does not automatically make it fast and memory efficient. Training techniques, packages, jobs and parameters must be adjusted accordingly to make full use of a systems capabilities.

### 4. GPU computation and why CUDA matters

GPUs are optimized for executing a large volume of operations in parallel, because they can apply the same calculation to thousands of data points at the same time. In comparison, CPUs are designed to switch quickly between different tasks instead of processing large amounts of data in parallel.

CUDA is the programming and execution model that allows frameworks, such as machine learning tools PyTorch or Tensorflow, to schedule work on a GPU. When a ML model is moved to a CUDA device, all tensor operations<sup>2</sup> are translated into GPU kernels: small programs that are put on the GPU and run the same calculation for many numbers. The kernels are launched asynchronously (async) by the CPU “giving work” to the GPU and then continuing executing its own code without waiting for the GPU calculations to finish. This allows data preparation/loading and GPU calculations to happen at the same time, massively increasing speed.

---

<sup>1</sup> Floating point operations: Individual calculations computers perform on numbers with decimal points.

<sup>2</sup> Tensor operations: Groups of calculations performed on many numbers, often organized in arrays, at the same time.

## 5. Mixed precision: doing less work without losing accuracy

Most neural network computations are based on multiplying matrices<sup>3</sup>. The numbers in these matrices are stored using floating point (decimal) formats, where the amount of bits means how precise a value can be. For example, a 32-bit floating point number stores more digits and is more precise in calculations, while a 16-bit floating point number stores less digits and so has less precision.

When training a network, many calculations do not actually need the full precision of 32-bit numbers. This is where mixed precision training comes in. Mixed precision works by performing most calculation using only 16-bit numbers, which are faster and require less memory than 32-bit numbers. During the forward pass in a neural network<sup>4</sup> large multiplications are executed using 16-bit precision, because these small differences in numbers do not affect the final predictions. Some calculations however must stay in 32-bit precision, for example where many small numbers are combined in a single value (e.g. when computing the loss). If this was done in 16-bit precision, rounding could change overall training.

By using the mixed precision technique, memory usage is much more efficient and overall speeds up training without reducing any accuracy.

---

<sup>3</sup> Matrices: Large tables of numbers

<sup>4</sup> Forward pass: Step where input data is passed through the neural network to produce predictions.

## 6. Parallel data loading

Before data can be used for calculations on a GPU, it must first be read from disk and prepared by the CPU. This stage is called data loading. To speed this up, programs use multiple worker processes. A worker is a separate CPU process with only one job to load and prepare data.

These workers run in parallel and the amount of workers is manually defined. The number of workers that can be used depends on the type of CPU. Each worker needs CPU time, and most CPU's are divided into multiple cores. These cores allow multiple processes to run at the same time. Setting a larger number of workers than available CPU cores means they have to share cores, which decreases efficiency and adds a lot of extra time.

However there is a limit when using multiple workers. Large datasets often contain many small files. When that's the case, opening every file and accessing file information consists of a very large number of processes. When too many workers are active at the same time, they have to “compete” for accessing the filesystem, which can increase the time it takes to load the data even though there are many workers.

## 7. Keeping the CPU and GPU busy

A GPU can only perform calculations on data that is stored in its own Random Access Memory (RAM), on a GPU this is called Video Random Access Memory (VRAM). Image files are initially stored on disk (eg. hard drive on the computer). Before the GPU can actually use the data, it must go over a few steps:

1. The image file is read from disk into the computer's main RAM, which is used by the central processing unit (CPU).
2. The CPU decodes the image by converting the compressed file format into numbers, and if needed apply transformations such as resizing.
3. The numerical data is copied from system RAM into the GPU's VRAM.

Pinned (page-locked) memory helps speed up step 3. Normally, the operating system is allowed to move data around in system RAM while programs are running. Because the data might move, it first has to be copied into a temporary location before it can be sent to the GPU, which adds extra time for copying. When memory is pinned, the data is in one place in system RAM. This allows the GPU to copy the data directly into VRAM using Direct Memory Access (DMA). By using this, the GPU gets new data faster and spends less time waiting.

However, only using pinned memory is not enough to keep data transfer from slowing down the whole process. When data is copied in a synchronous (blocking) way, the transfer to the GPU starts and then the CPU waits until the copy is finished before continuing anything. This way the CPU cannot prepare the next batch of data yet.

With a asynchronous (non-blocking) transfer, the CPU starts the transfer of data and can also immediately execute other code. When the data is still being transferred to the GPU, the CPU can already prepare the next batch of data. This reduces waiting time and helps to keep both the CPU and GPU busy all the time.

## 8. Reducing unnecessary work

The next thing that could limit performance speed is the additional ‘work’ that is often done in training scripts. Specifically work that is not actually part of the neural networks’ computations. When training a neural network, thousands to millions of steps are constantly being repeated. Any small amount of extra work inside the training loop can increase these steps by a lot.

A few examples of extra work that often happens in training code are clearing gradients<sup>5</sup>, tracking and updating metrics, and logging and/or visualizations.

### 8.1. Clearing gradients

During training of a neural network, it learns by computing gradients, they describe how each parameter should change to improve learning. These gradients are calculated over and over again during each training step. Before any new gradients can be calculated, the old ones must be removed. A common method for this is to set all gradients to zero using the function `optimizer.zero_grad()`. This function writes zeros in memory for every parameter at every training step, which is a large amount being written to memory.

A more efficient method is to mark the gradients as ‘empty’ using a small modification to the previous function: `optimizer.zero_grad(set_to_none=True)`. After using this, then new gradient values are created automatically during the next step. This avoids unnecessary memory writing and makes each step faster, especially for larger models that have more gradients to update.

### 8.2. Tracking and updating metrics

During training, it is common to track metrics (e.g. accuracy, loss, recall etc..) to monitor how well the neural network is learning. To compute these metrics, the predictions are compared with the correct labels. This work is usually done on the CPU because the calculations only involve comparing results and counting values. Using the GPU for this would take time away from the main computations used in training. However, it does require extra time to copy data from the GPU back to system memory.

These metric calculations are not part of the networks’ calculations itself, and do not influence how the model learn. but without metrics it is not possible to know how well learning is going. For this reason, it is important to find a balanced metric setup.

When metrics are calculated very often, for example after every batch or straining step, it can slow down the training because the GPU has to wait before starting the next step. For large scale training it is more efficient to calculate metrics less often, for example once per epoch or just on a smaller subset of data. This keeps the training loop more focused on

---

<sup>5</sup> Gradients: Values that describe how each parameter in a network should change to reduce prediction mistakes.

meaningful metrics, and the calculation of these metrics doesn't slow down overall performance.

### 8.3. Logging and visualization

Logging and visualization are used, together with the metrics, to keep track of the training progress. Examples of this are writing metrics to log files or online dashboards for AI training, saving network checkpoints, or generating plots or matrices for metrics. These all take up CPU time and access to storage, which is much slower than computations done on the GPU.

To reduce this extra time, logging and visualizations should be moved outside of the main training loop, so it does not consistently happen during training. Logging metrics can temporarily be stored in memory and later written to log files, as well as the visualizations can be created at the end when training finishes.

## 9. Memory usage and batch size

When training neural networks on HPC systems, memory usage is an important and limiting factor. During training, data is used in groups of data called batches. The batch size defines how many input samples are used at the same time during a training step. Batch size is one of the most important parameters when training on HPC systems, because it massively influences both the training speed and memory usage.

Using larger batch sizes makes the GPU perform more calculations in parallel, which improves speed. However larger batch sizes also require more memory because results for each batch must be stored for every sample in the batch until the training step is finished. Memory usage is highest a step called backwards pass<sup>6</sup>. During this step the networks calculates how to update the parameters. To do this, the network needs to access the intermediate results of the forward pass. These results must be kept in memory until the backwards pass is finished. As a results, using a larger batch size increases memory usage for not only the data but also for intermediate results.

Batch size is often the first parameter that causes training to run out of memory on HPC systems (large batch size) or slow down learning (smaller batch size). However, there is not one set formula for which batch size is best to use. Analysis shows that besides influencing memory and speed, the batch size value can also influence the actual accuracy of a network (Jastrzębski, 2018; Masters & Luschi, 2018), because it sees a different volume and combination of the data in each step.

Batch sizes are usually factors of two (e.g. 16, 32, 64, 128 etc.), because computers store data in binary, and powers of two fit into memory without needing additional handling.

---

<sup>6</sup> Backwards pass: Step where the network ‘takes a step back’ to look at results of the forward pass and update the gradients.

## 10. Job configuration

On HPC systems, resources like GPU type, CPU cores, memory and maximum runtime have to be configured before a training job starts. Once a job is running these resources are reserved for the job, even if they are not used. When requesting too little resources the job can fail, and when requesting too many a large number of resources are blocked for other people to use (and your colleagues using the same HPC won't be happy).

Requesting more GPUs or memory than the training actually needs does not increase performance. If the code is written for a single GPU training, adding additional GPUs will not change anything. The number of CPU cores chosen must also match the settings defined in the training code (as explained in Chapter: [Parallel data loading](#)). When requesting too little CPU cores, data loading can become very slow. When requesting too many CPU cores, a large part of these cores might not be used. Because these cores are reserved for the job, they cannot be used by any other jobs on the HPC system.

HPC systems usually offer different types of GPUs. These GPUs differ in how fast they can perform calculations and how much memory they have. Some GPUs have small amounts of memory and are used for training smaller networks, while others have much more memory and are needed for large models or larger datasets. Choosing a GPU with too little memory can break the training and exit the script.

### 10.1. Scheduling

HPC systems use job scheduling systems to manage multiple people running multiple jobs and requesting resources. Instead of running an uploaded script immediately, jobs are usually placed in a queue. The scheduler starts jobs when the requested resources from the job configuration become available. This ensures that many users can use the system at the same time without interfering with each other. A very common job scheduling system is Simple Linux Utility for Resource Management (SLURM). It runs on Linux based HPC systems and has easy commands that let admins control job queues and let users control their resources and jobs running.

## 11. Inference at large scale

Inference is the process of using a trained neural network to generate predictions on new data. Unlike training, inference only performs the forward pass of the network. No gradients are computed, and model parameters are not updated.

Because there is no backward pass, inference is not limited by the calculations it has to do, but by the data loading and data transfers. If data loading is slow the GPU will remain waiting even though inference itself is fast.

Batch size has a different function during inference than during training. For inference larger batch sizes mean that more samples are processed per second. Since inference doesn't affect any learning, very large batch sizes are fine as long as they fit into available memory. The optimal batch size for inference is often larger than the batch size used during training.

Large scale inference must however also be stable. Unlabeled inference datasets sometimes have corrupted files, missing data or unexpected formats. If a single corrupt input fails the entire job, a large amount of resources is wasted. Therefore it is important to include checks and safe handling for any errors in the input data, so that inference can continue smoothly.

## 12. Reproducibility and stability

Running machine learning setups on HPC systems can introduce randomness that is not present on smaller setups. Parallel execution, asynchronous operations, and multi-process data loading can cause small differences between runs, even when the exact same code and datasets are used.

Reproducibility is important for debugging, comparison of experiments and most importantly validation of results. Setting a fixed random seed helps reduce randomness. A random seed is a value used to generate random numbers. Many steps in training a network require randomness, for example shuffling the training data, initializing model parameters, or applying random augmentations. When the same random seed is used for those operations, these ‘random choices’ are made in the same order each time the code is executed.

The seed must be applied across all parts of the training setup, including the programming language, libraries and the deep learning framework itself. If only part of the code uses a seed, other components can still introduce randomness. On HPC systems this is important since training often uses multiple worker processes and external libraries which can all generate random values.

Using a fixed random seed makes experiments more comparable. However, even with fixed seeds, identical results are still not always guaranteed on HPC systems. Some GPU operations are executed in parallel and asynchronously, which can also cause small numerical differences between runs.

Besides setting random seeds, reproducibility can be improved by limiting non-deterministic execution paths<sup>7</sup>. On GPUs some libraries automatically select faster algorithms that perform calculations in parallel, where the order of how intermediate results are combined is not always the same. Because floats are rounded during calculations, changing this order can also lead to small numerical differences. By forcing deterministic<sup>8</sup> computations, the same calculations are executed in the same order every single time. This can reduce speed a little bit, but it improves consistency and makes experiments easier to compare.

Stability is therefore just as important as reproducibility. On HPC systems, jobs can be interrupted by time limits, maintenance, hardware failures etc.. Without proper handling, this can result in lost progress and wasted resources.

Finally, checkpointing is a method used to save the current state of a training run. A checkpoint often contains the models’ parameters, optimizer state, and/or information about the current training step or epoch. By saving this information, training can later be resumed for the exact same point.

---

<sup>7</sup> Non-deterministic: The same operation can be executed in a different order between runs.

<sup>8</sup> Deterministic: The same operation is executed in the same order every time it runs.

## 13. The environment and conclusion

Training and running machine learning models on HPC systems consumes a large amount of electrical energy. This energy is used not only by GPUs and CPUs, but also by memory, storage, and cooling infrastructure. Inefficient use of HPC resources directly increases energy consumption and a larger carbon footprint.

Unnecessary energy usage mostly comes from unused or waiting resources. GPUs use power even when they are assigned to a job but not currently doing calculations. This can happen when data loading is slow, batch sizes chosen incorrectly, or extra work is performed inside training or inference loops. When GPUs spend time waiting, energy is used without producing useful results.

Reducing total runtime reduces energy usage. Techniques discussed in previous chapters, like mixed precision, efficient data loading, and correct job configuration, reduce the total number of GPU hours needed. Finishing jobs faster = lower overall energy consumption.

For large-scale inference, avoiding repeated or unnecessary computation is important. Re-running inference on the same data, or failing on single corrupted inputs, wastes resources. Stable scripts and basic error handling prevent this.

We cannot always control how a HPC system generates energy, but we can control how allocated resources are used. Writing efficient code, requesting the right resources, and avoiding unnecessary computation directly improves overall training efficiency and reduces the environmental impact of AI workloads on HPC systems.